BlendMAS: A BLockchain-ENabled Decentralized Microservices Architecture for Smart Public Safety

Ronghua Xu, Seyed Yahya Nikouei, Yu Chen
Binghamton University, SUNY
Binghamton, NY 13902, USA
{rxu22, snikoue1, ychen}@binghamton.edu

Erik Blasch, Alexander Aved
US Air Force Research Laboratory
Rome, NY 13441, USA
{erik.blasch.1, alexander.aved}@us.af.mil

***Abstract*—Thanks to rapid technological advances in the Internet of Things (IoT), a smart public safety (SPS) system has become feasible by integrating heterogeneous computing devices to collaboratively provide public protection services. While service-oriented architecture (SOA) has been adopted by IoT and cyber-physical systems (CPS), it is difficult for a monolithic architecture to provide scalable and extensible services for a distributed IoT based SPS system. Furthermore, traditional security solutions rely on a centralized authority, which can be a performance bottleneck or single point failure. Inspired by microservices architecture and blockchain technology, this paper proposes a BLockchain-ENabled Decentralized Microservices Architecture for Smart public safety (BlendMAS). Within a permissioned blockchain network, a microservices based security mechanism is introduced to secure data access control in an SPS system. The functionality of security services is decoupled into separate containerized microservices that are built using a smart contract and deployed on edge and fog computing nodes. An extensive experimental study verified that the proposed BlendMAS is able to offer a decentralized, scalable and secured data sharing and access control to distributed IoT based SPS system.**

***Index Terms*—Blockchain, Microservices Architecture, Smart Contract, Internet of Things (IoT), Smart Public Safety (SPS).**

I. Introduction

The proliferation of Internet of Things (IoT) technology allows the concept of Smart Cities to become feasible with smart surveillance as one of the most intensively studied topics in the IoT community. Smart Public Safety (SPS) systems process surveillance video streams at the edge and utilize many smart sensors. However, there are still challenges to be tackled in order to realize a fully functional IoT-based SPS system in practice. Relying on a centralized architecture that is based on cloud computing center inevitably adds uncertain latency and poses extra workload to communication networks. While merging lower-level image processing tasks with an edge computing platform is able to meet the requirements for delay-sensitive, mission-critical applications [9], [23], new challenges are also introduced by the distributed and cross-domain features such as scalability, heterogeneity and interoperability.

The SPS system is deployed in a distributed network environment that includes a large number of IoT devices (camera + edge hardware) with high heterogeneity and dynamics. The heterogeneity and resource constraint at edge necessitate a scalable, flexible and lightweight system architecture that supports fast development and easy deployment among multiple service providers. Furthermore, those smart devices are geographically scattered across near-site network edges. It is therefore not suitable to enforce security policies on a centralized authority basis, which suffers from the performance bottlenecks or single point of failures. Thus, the SPS system needs a decentralized framework that provides a security mechanism in the trust-less network environments.

Recently, a novel service oriented architecture (SOA), called the microservices architecture, has emerged and gained a lot of popularity [16] in designing a smart city platform. Instead of deploying the system as a monolithic unit as traditional SOAs do, the microservices architecture divides an monolithic application into multiple atomic microservices that run independently on distributed computing platforms. Each microservice performs one specific sub-task or service and requires lightweight communication with other system components. Such characteristics make the microservices architecture an ideal candidate to build a flexible platform, which is easy to be developed and maintained for cross-domain applications. Specifically, the microservices architecture possesses many attractive features, such as good scalability, fine granularity, loose coupling, continuous development, low maintenance cost, and so on. These beneficial features make a microservices architecture a natural selection to enhance SPS systems based on the edge computing paradigm.

A microservices-enabled application also demonstrates vulnerabilities in security due to its usage of distributed data sharing and accessing interfaces [38]. Recent methods demonstrate efforts in developing new decentralized security solutions for distributed network applications. Blockchain, which acts as the fundamental protocol of Bitcoin [19], has demonstrated great potential to revolutionize the fundamentals of information technology (IT) due to many attractive properties, such as decentralization and transparency. Decentralized Application (DApp), which is built on smart contract and deployed on blockchain network, performs pre-defined algorithms and agreement without relying on third-party intermediary. Blockchain and smart contract together are promising to provide a decentralized solution to enable a secured data sharing and access control for SPS systems.

In this paper, a BLlockchain-ENable Decentralized Microservices Architecture for SPS (BlendMAS) is proposed

978-1-7281-4693-5/19/$31.00 ©2019 IEEE
DOI 10.1109/Blockchain.2019.00082

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:37:47 UTC from IEEE Xplore. Restrictions apply.

to secure data accessing among different service providers and entities in a public safety system. The proposed platform follows the divide-and-conquer principle to decouple the SPS and security functionality into multiple containerized microservices that are computationally affordable to each individual computing platform. The distributed microservices could cooperate with each other as a service pool to perform complicated decision-making and analytical missions. The mining services enforce a consensus mechanism among a large number of authorized miners to maintain sanctity of the data recorded on the permissioned blockchain network. The security mechanism of the SPS is implemented as separated microservices that are built on the smart contract. The hash of the frame features and the corresponding decision is put into a block data, that is approved and appended to the blockchain network for data tampering prevention. The identity authentication and access control strategy ensure that only an authorized entity is capable of accessing services and data in a SPS system.

The major contributions of this paper are:

1) A complete architecture of microservice-based SPS platform is introduced, which is implemented in a hierarchical edge-fog-cloud computing paradigm;
2) A fully functional permissioned blockchain network is implemented as microservices and deployed on a physical network;
3) A prototype of smart contract enabled data sharing and access control mechanism is designed and tested on a permissioned blockchain network; and
4) A comprehensive experimental study has been conducted that compares the proposed BlendMAS with the monolithic SOA framework. The experimental results validate the feasibility of the BlendMAS scheme in IoT environments without introducing significant overhead.

The remainder of this paper is organized as follows: Section II analyzes and reviews the state of the art research and ongoing effort in each of components adopted in a SPS system. Section III illustrates the details of the proposed BlendMAS system. Beside the implementation of the proof-of-concept prototype, Section IV reports an extensive experimental study using test scenarios that are built on both edge devices (Raspberry Pi) and fog computing devices (Desktop). Finally, a summary is presented in Section V.

II. Background Knowledge and Related Work

A. Smart Public Safety

Traditional surveillance systems depend on human operators to interpret the processing of captured video [8]. However, there is a growing demand for human resources to monitor the data stream as the camera numbers rise in congested areas [6]. Recently a number of smart systems are introduced which aim at minimizing the role that human operators play in object detection, such that the responsibility of abnormal behavior detection is taken by various more intelligent machine learning (ML) algorithms [29]. Some techniques employ statistical analysis [13] or [26] that use more modern ML approaches where the algorithm automatically processes the collected video frames in a cloud to detect, track, and report any unusual circumstances.

These traditionally algorithms are computationally expensive and normally implemented at the powerful cloud servers of the surveillance system. An example is the Wide Area Motion Imagery (WAMI) that transforms the frames from the image sensors back to the cloud for processing [11], [30], [31], [32]. Earlier studies show that this approach puts a heavy burden on the network [9], [10]. Ideally, the minimum delay and communication overhead is achievable if all the functions are conducted on-site at the network edge, and the decision is made instantly.

Recently, the smart surveillance community has introduced some decentralized surveillance frameworks that are more convincing in many mission-critical, delay sensitive tasks [20], [37]. Implemented based on the edge-fog-cloud hierarchy architecture. the input frame that is streamed out of the surveillance camera is given to an edge unit where low-level processing is performed [21], [22]. The intermediate-level is fog nodes, where multiple tasks are performed based on the processing power and resources available. Finally, the cloud is focused on historical profile building, algorithm fine tuning, and global statistical analysis, which depends on the type of decisions the system is going to make.

B. Microservices in IoT

A service oriented architecture (SOA) is widely adopted in the development of application software in a IoT and CPS environment [7]. The traditional SOA utilizes a monolithic architecture that constitutes different software features in a single interconnected and interdependent application and database. Owing to the tightly coupled dependence among functions and components, such a monolithic framework is difficult to adapt to new requirements in an IoT-enabled system, such as scalability, service extensibility, data privacy, and cross-platform interoperability [12]. As an extension of the traditional SOA, the *microservices architecture* allows functional units of an application to work independently with a loose coupling though encapsulating a minimal functional software module as a microservice, which can be individually developed and deployed. Each microservice is a process dedicated to certain function of the application. The individual microservices communicate with each other through a lightweight mechanism, such as HTTP RESTful API or a message bus asynchronously [17]. Finally, multiple decentralized individual microservices cooperate with each other to perform the functions of complex systems. The flexibility of microservices enables continuous, efficient, and independent deployment of application function units. As two significant features of the microservices architecture, *fine granularity* means each of the microservices can be developed in different frameworks and with minimal development resources, while *loose coupling* implies that functions of microservices components is independent of each others deployment and development [38].

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:37:47 UTC from IEEE Xplore. Restrictions apply.

Thanks to granularity and coupling properties, the microservices architecture has been investigated in many smart solutions to enhance the scalability and security of IoT-based applications. The IoT systems are advancing from "things"-oriented ecosystems to a widely and finely distributed microservices-oriented ecosystems [12]. An Intelligent Transportation Systems (ITS) that incorporates and combines the IoT approaches using the serverless microservices architecture has been designed and implemented to help the transportation planning for the Bus Rapid Transit (BRT) systems [15]. To enable a more scalable and decentralized solution for advanced video stream analysis for large volumes of distributed edge devices, a conceptual design of a robust smart surveillance systems was proposed based on microservices architecture and blockchain technology [18]. It aims at offering a scalable, decentralized and fine-grained access control solution for smart surveillance systems.

C. Blockchain and Smart Contract

As a fundamental technology of Bitcoin [19], *blockchain* initially was used to promote a new cryptocurrency that performs commercial transactions among independent entities without relying on a centralized authority, like banks and government agencies. Essentially, the blockchain is a public ledger based on consensus rules to provide a verifiable, append-only chained data structure of transactions. Thanks to the decentralized architecture that does not rely on a centralized authority, blockchain allows the data to be stored and updated distributively. The transactions are approved by miners and recorded in the time-stamped blocks, where each block is identified by a cryptographic hash and chained to preceding blocks in a chronological order. In a blockchain network, a *consensus mechanism* is enforced on a large amount of distributed nodes called miners to maintain the sanctity of the data recorded on the blocks. Thanks to the trustless proof mechanism running on miners across the network, users can trust the system of the public ledger stored worldwide on many different decentralized nodes maintained by "miner-accountants", as opposed to having to establish and maintain trust with a transaction counter-party or a third-party intermediary [27]. Thus, blockchain is an ideal decentralized architecture to ensure distributed transactions among all participants in a trustless environment, like edge-based IoT networks.

Emerging from the intelligent property, a *smart contract* allows users to achieve agreements among parties through a blockchain network. By using cryptographic and security mechanisms, a smart contract combines protocols with user interfaces to formalize and secure relationships over computer networks [28]. A smart contract includes a collection of pre-defined instructions and data that have been saved at a specific address of blockchain as a Merkle hash tree, which is a constructed bottom-to-up binary tree data structure. Through exposing public functions or application binary interfaces (ABIs), a smart contract interacts with users to offer the predefined business logic or contract agreement. The blockchain and smart contract enabled security mechanism for applications has been a hot topic and some efforts have been reported recently, for example, smart surveillance system [18], [25], social security system [36], space situation awareness [35], identification authentication [14] and access control [33], [34]. Blockchain and smart contract together are promising to provide a solution to enable a secured data sharing and access authorization in decentralized SPS systems.

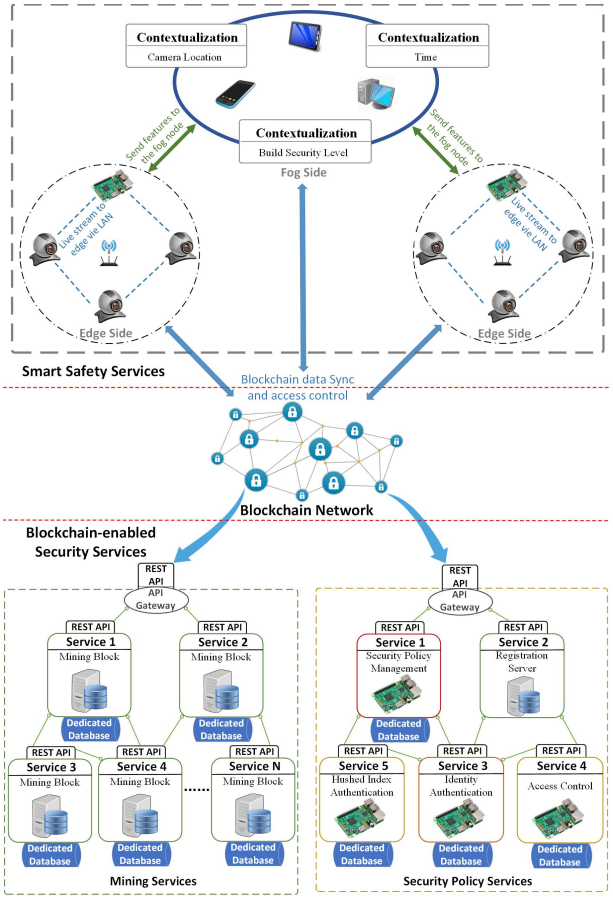

Fig. 1. Illustration of the BlendMAS System Architecture.

III. System Design of BlendMAS

Leveraging the attractive characteristics of the microservices architecture that support fine granularity, loose coupling, and continuous delivery, the BlendMAS system is a completely decentralized solution where individual functional components of system are developed by different teams and hosted by heterogeneous hardware platforms, as shown in Fig. 1. In the system design, the Docker container is adopted for the microservices architecture and the multi-layer BlendMAS platform is implemented following the edge-fog-cloud computing paradigm. Two type of containers are deployed at the edge layer. One is responsible for the security policy service that enforces the data access control and verification to prevent from unauthorized service request and data tampering, while another is the video stream processing microservice to extract features of frames. Owing to more powerful computing and

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:37:47 UTC from IEEE Xplore. Restrictions apply.

storage resources, the features fusion, behavior analysis and mining microservices are hosted at the fog layer or cloud layer.

A. System Architecture of BlendMAS

Figure 1 illustrates the proposed BlendMAS system architecture, which utilizes microservices-enabled private blockchain network to secure video stream services while providing secured data sharing. The proposed system consists of three services:

- *Smart Surveillance Application Services*: These services provide functions to support smart surveillance, such as video stream processing, object detection and tracking, and movement features extraction. Real-time video streams are generated by cameras and transmitted to edge microservices for features extraction. Lower level features are transferred to fog nodes for data aggregation and higher level analytic services, such as pattern recognition, behavior analysis and anomalous event detection.
- *A Permissioned Blockchain Network*: The security microservice provides not only network communication channel on the Internet, but also a private blockchain network infrastructure running on a decentralized peer-to-peer network. All the network communications among entities run on TCP/IP protocol. Security solutions, such as identity authentication and access control, are developed as Decentralized Application (DApp) which is based on a smart contract and deployed on the blockchain network.
- *Blockchain-enabled Security Services*: The security service section acts as a fundamental service pool to support key functions of security mechanism. The provided services could be divided into two main clusters: mining services and security policy services. As a core function of maintaining the blockchain network, mining services are responsible for executing consensus algorithms to verify transactions and generating new blocks. The miners are containerized microservices running on single or multiple host machines to fulfill mining task independently. Finally, multiple certificated miners cooperate with each other to secure the private permissioned blockchain network. All the security polices and models, such as identity authentication and access control, are transcoded into separate microservices, and those microservices work together as a security policy services cluster. Through implementing each security model or policy as a single microservice that works independently from each other, the security policy services cluster could address scalability and heterogeneity in IoT-based smart surveillance systems by offering more flexible, interoperable and lightweight security solutions.

In the SPS system, the feature data extracted by the edge computing will be merged with contextual data on fog layer nodes for high level tasks (such as situation awareness). Therefore a security mechanism is necessary to protect the data shared among functional service nodes and enforce an access control policy. To enable a decentralized, scalable, and fine-grained security scheme for SPS system, the proposed BlendMAS is focused on two issues: the permissioned blockchain management, and security mechanism enforcement.

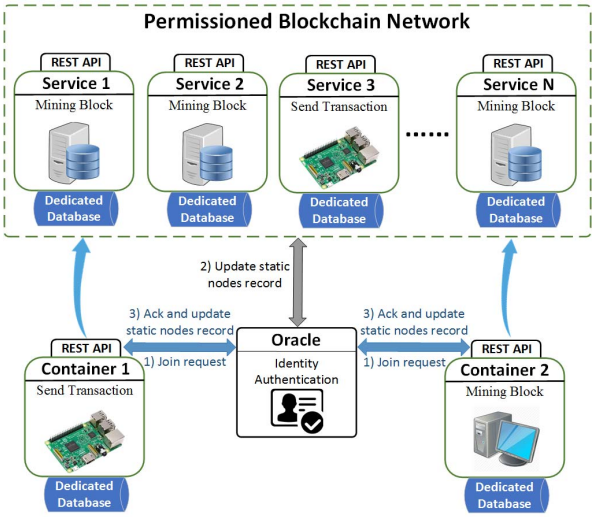

Fig. 2. Identity-based Permissioned Blockchain Network.

B. Identity-based Permissioned Blockchain

All the entities on the permissioned blockchain network are implemented as containers, which perform blockchain services independently on the host machines. The containerized microservices could be categorized as miners or non-mining nodes given the computation power of the host machines. Only the authorized participants could be recognized by entities of network and perform blockchain services, such as mining blocks, sending transactions and deploying smart contracts. An identity management mechanism ensures that participants identify each other while not necessarily fully trust each other.

Figure 2 illustrates the identity authentication process to enroll new node in the permissioned blockchain network. An oracle, who acts as the administrator of blockchain network, maintains a global node registration and identification policies for the permissioned blockchain network management. To join the permissioned blockchain network, the participants must send joining requests to an oracle for identity authentication. The oracle verifies new entity's joining request by performing identification policies. After the oracle approved the joining requests, the new entity's node information will be added to a global static node record, and the oracle will send updated static nodes record to all certificated participants in blockchain network accordingly. The membership revocation occurs when an entity explicitly launches leaving request or oracle implicitly rule out any misbehaved node. The oracle simply updates static node record on each participant to change configuration of blockchain network. As shown in Fig. 2, all the containerized miners cooperate with each other to enforce consensus mechanism, while other non-mining containers function as service providers to offer blockchain

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:37:47 UTC from IEEE Xplore. Restrictions apply.

interactive services like sending transactions. Compared to a public blockchain network, the permissioned blockchain network can achieve a more efficient consensus mechanism, and more secured network by limiting participants with clearly defined security policies.

C. Blockchain-enabled Security Microservices

Utilizing the microservices architecture, the security functions are decoupled into multiple microservices and deployed on distributed computing devices. These decentralized security microservices work as a service cluster to offer a scalable, flexible and lightweight data sharing and access control mechanism for the SPS system. The key service components and operations are introduced below.

1) Registration Service: In the proposed BlendMAS system, all entities must sent registration request to registration microservice before accessing smart surveillance services. Entity registration process is performed by the registration microservices associating entity's unique blockchain account address with a Virtual ID (VID). All the registration information indexed by VID are recorded in a profile database maintained by the registration microservices on the fog node.

2) Identity Authentication: Since each blockchain account is uniquely indexed by its address that is derived from his/her own public key, the account address is ideal for identity authentication needed by other security microservices, such as hashed index recording and access control. The identity authentication microservices expose RESTful API to other microservices-enabled providers for referring identity verification results. Once an authentication service request is received, the identity authentication decision making process queries the requester identity profile from the registration service, and returns the identity verification results according to the predefine policies.

3) Security Management: In the security services cluster, the security management microservices act as data and security service managers who deploy the smart contracts encapsulating hashed index authentication and the access control policies. Taking advantage of the cryptographic and security mechanisms provided by the blockchain network, smart contracts can secure any algorithmically specifiable protocols and relationships from possible malicious interference by third parties in a trustless network. After the smart contracts have been deployed successfully on the blockchain network, they become visible to the entire network. The authorized participants could call Remote Procedure Call (RPC) interfaces to interact with smart contract.

4) Hashed Index Authentication: To successfully save the generated hashed index record to the blockchain, a hash index recording microservices entity initially sends an access request to the security management microservices to get a permission for executing the hashed index record generation application binary interfaces (ABI) functions of a smart contract. If the access request is granted, the index recording microservices receive acknowledgement from security management microservices with a smart contract address and the ABI function for recording hashed index data. After generating hashed index records, the hashed index recording microservices simply interact with the authorized ABI function to save the hashed index data to the blockchain. In hashed index authentication process, microservice queries a hashed key-value index by interacting with smart contract and compares it with calculated hash values of the record index table. Finally, the authentication results are sent back to service requester.

5) Access Control: To successfully access services or resources at SPS service providers, an entity initially sends an access right request to the access control microservices to get a capability token. Given a reference of the entitys profile, which is the authenticated identity information maintained by the registration microservices, a decision making policy module running on the access control microservices evaluates the access request by enforcing the authorization policies. If the access request is granted, the access control microservice issues the capability token encoding authorized access right, and then launches a transaction to update the token data in the smart contract. Once the transaction has been approved and recorded in a new block, the access control microservice responds to the requester by providing a smart contract address for the querying token data. Otherwise, the access right request is rejected and a denied acknowledgement is returned. Authorization validation process is triggered when when a smart surveillance service provider receives a service request from users. Given the validation result that demonstrates the access right policies and conditional constraints are satisfied, the service provider grants the access request and offers services to the requester. Otherwise, the service request is denied.

IV. Experimental Analysis

The BlendMAS system is actually the security sub-system of a complete SPS system. Due to the limited space, the implementation and experimental evaluation of the entire SPS prototype are not reported here. But interested readers may find the experimental results of the microservices architecture based video stream processing at the edge in [24].

A concept-proof prototype system has been implemented on a real private Ethereum [1] blockchain network environment. The smart contract development use Solidity [4], which is a contract-oriented, high-level language for implementing smart contracts. In order to evaluate the performance and the overhead of our proposed access control scheme, the security microservices have been implemented using docker container and deployed on both on edge and fog units. The web service application is based on the Flask framework [2] using Python, and the profiles and policy rules management are developed using an embedded structured query language (SQL) database engine, called SQLite [5].

A. Environmental Setup

The mining microservices are deployed on a platform with stronger computing power, like a laptop or a desktop. Two miners are deployed on a laptop and other four miners are distributed on four desktops. Table I describes configurations

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:37:47 UTC from IEEE Xplore. Restrictions apply.

of nodes used in the experiments. In this prototype, the laptop acts as a cloud computing server, which takes role of oracle to manage blockchain network. All desktops work as fog computing nodes, while a Raspberry PI 3 Model B runs as edge computing node. The SPS functions and security microservices are hosted both on fog and edge computing. All devices use Go-Ethereum [3] as the client application to work on the blockchain network.

TABLE I
CONFIGURATION OF EXPERIMENTAL NODES.

Device	Lenovo P50	Dell Optiplex 760	Raspberry Pi 3 Model B
CPU	2.3 GHz Intel Core i7 (8 cores)	3 GHz Intel Core TM (2 cores)	quad-core ARM Cortex A53, 1.2GHz
Memory	16GB DDR3	4GB DDR3	1GB SDRAM
Storage	250G SSD+ 500G HHD	250G HHD	32GB (microSD card)
OS	Ubuntu 16.04	Ubuntu 16.04	Raspbian GNU/Linux 8 (jessie)

B. Performance Evaluation

To evaluate the performance of the microservices-based security mechanism, a service access experiment is carried out on a physical network environment which includes 3 Raspberry PIs and 2 desktops. One Raspberry PI works as a client to send service request, while server side is SPS service provider, who has been both hosted on edge (Raspberry PI) and fog (desktop) nodes. A blockchain enabled capability based access control (BlenCAC) scheme [33] is selected to enforce the access control policies. The hashed index authentication microservice and access control microservice are deployed on two Raspberry PIs separately. To measure the general cost incurred by the BlendMAS scheme both on the edge device processing time and the network communication delay, 50 test runs have been conducted based on the proposed test scenario, where the client sends a data query request to server for an access permission. This test scenario is based on an assumption that the client has been assigned a valid token when it performs the action on server. Therefore, all steps of hashed index authentication and access right validation must be processed on the server side so that the maximum latency value is computed.

1) Computational Overhead: Figure 3 shows the computational overhead introduced by AC process. The entire executing time of the access control process is 42.4 ms (41.8 ms + 0.1 ms + 0.5 ms) on the edge device and 14.5 ms (14.2 ms + 0.1 ms + 0.2 ms) on the fog node. The average time for querying data from the smart contract is 41.8 ms on edge device and 14.2 on fog node. Since the authorization process includes CapAC token validation and the access right verification, the average time of authorization process is about 0.6 ms (0.1 ms + 0.5 ms) on the edge device and 0.3 ms

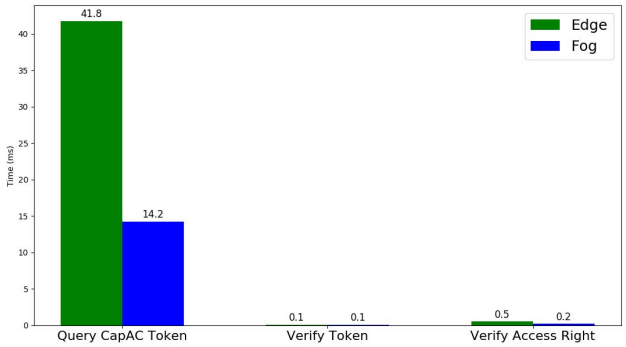

Fig. 3. Execution time of each individual stage of the access control Microservices.

(0.1 ms + 0.2 ms) on the fog node. Owing to cryptography and hash chain computations in data from smart contract, querying CapAC token introduces the highest overload among AC operation stages. The average total delay time cause by service request operation of retrieving data from client to server is 196 ms on the edge device and 85 ms on the fog node. Compared with total delay time, the overload introduced by AC enforcement is about 21 % (42.4 ms / 196 ms) on the edge side and about 17 % (14.5 ms / 85 ms) on the fog side.

Figure 4 shows computational overhead incurred by hashed index authentication on both edge and fog sides. Similar to access control process, a querying hashed index operation, which is mainly responsible for fetching hashed index value from smart contract, is the most computing intensive stage. Since the fog nodes have more computation capacity than the edge nodes do, while the execution time of querying hashed index token on edge nodes is about 23.4 ms, the same operation takes only 10.9 ms on fog nodes. The index authentication process is divided into two steps, extracting hashed value from the features data and verifying the hashed index from smart contract given hashed value. The average time of the authorization process is about 1.8 ms (1.4 ms + 0.4 ms) on edge nodes and 0.8 ms (0.6 ms + 0.2 ms) on fog nodes. The average total delay time introduced by the service request operation of retrieving data from client to server is 161 ms on edge device and 51 ms on fog node. Compared with total delay time, the overload introduced by hash index authentication is about 14 % (23.4 ms / 161 ms) on the edge side and about 21 % (10.9 ms / 51 ms) on the fog side.

2) Communication Overhead: To evaluate the overall network latency incurred by the microservices architecture, a test has been done on two service architectures: the microservices architecture and the monolithic framework. Micro_App uses microservices framework in which service providers, BlednCAC and hashed index authentication, are decoupled into three separate containers and deployed on different host machines. While Mono_App uses monolithic framework that all service functions are encapsulated as one container and run on single host machine. Simulating a regular service request allows measuring how long it takes for the client to send a request and retrieve the data from the server.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:37:47 UTC from IEEE Xplore. Restrictions apply.

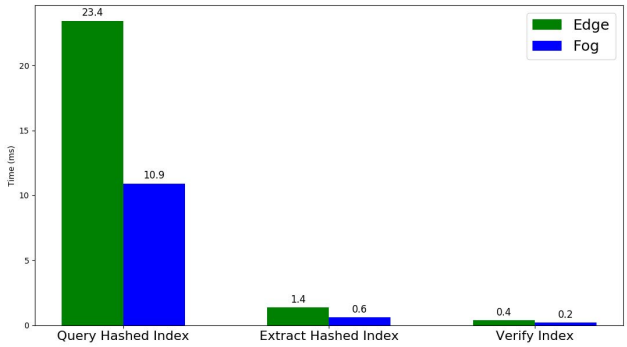

Fig. 4. Execution time of each individual stage of the Hashed Index Authentication Microservices.

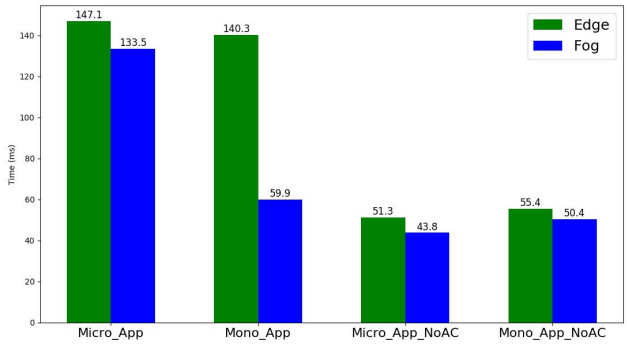

Fig. 5. The Network latency incurred by Microservices-enable BlendCAC Solution.

Figure 5 shows the overall communication latency incurred and compares the execution time of the BlendCAC and a benchmark without any access control enforcement on two service architectures. For the monolithic framework, at the fog, the benchmark without access control enforcement (Mono_App_NoAC) takes an average of 50.4 ms for fetching requested data versus that the BlendCAC (Mono_App) consumes on average of 59.9 ms. It means that embedding the BlendCAC scheme only introduces about 9.5 ms extra latency. At the edge devices, the Mono_App takes on average of 140.3 ms for fetching requested data versus Mono_App_NoAC on average of 55.4 ms, which means that access control process incurs 84.9 ms extra latency. For a microservices architecture, at the fog, the Micro_App incurs 89.7 ms (133.5 ms - 43.8 ms) extra latency compared with average time consumed by Micro_App. While at the edge devices, the Micro_App incurs 95.8 ms (147.1 ms - 51.3 ms) extra latency compared with Micro_App. No matter the Mono_App or Micro_App, the network latency incurred by embedding the BlendCAC on edge device is much higher than on fog node.

Considering the scenarios without access control enforcement, microservices and monolithic framework have almost the same performance both on edge and fog platforms. However, when it comes to BlendCAC scenario, the experiments on two architectures show different communication latency between fog and edge platforms. On the fog computing level, the network latency of Micro_App takes an average of 133.5 ms for fetching requested data versus that the Mono_App only consumes on average of 59.9 ms. But for edge devices, both frameworks have almost the same network latency, which is about 140 ms. Since services on monolithic framework utilize an Internal Process Communication (IPC) to call functions and share data, given the same computation and memory power, it is much more reliable and faster than the Remote Process Communication (RPC) used by microservices architecture, which is subject to the Quality of Service (QoS) determined by the network communication status. Since the fog nodes are more powerful than edge devices, the influences caused by computation and network communication for Micro_App is more significant than the same case for Mono_App on fog nodes. Although microservices incurs more network latency, which is 73.6 ms (133.5 ms - 59.9 ms) on the fog side and 6.8 ms (147.1 ms - 140.3 ms) on edge side, it still brings benefits to the distributed IoT-based system, such as loosely coupled dependence, easy service deployment and cross-platform interoperability.

C. Discussions

The experimental results demonstrate that the proposed BlendMAS strategy is effective and efficient to provide secured data sharing and access control services for IoT-based SPS system. Compared to centralized security solutions based on monolithic framework, the BlendMAS scheme has the following advantages:

1) *Decentralized security mechanism*: leveraging the blockchain and smart contract technology, the proposed BlendMAS scheme allows service providers to control their devices and resource without relying on a centralized third authority to establish the trust relationship with unknown nodes; the risk of performance bottleneck and the single point of failure incurred by centralized architecture are mitigated; and
2) *Fine-grained architecture*: embracing fine-grained microservices architecture enables the smart public safety toward domain-driven design, technology heterogeneity and continuous delivery, which is critical to the scalable, heterogeneous, flexible and dynamic IoT-based smart surveillance applications.

V. Conclusions

In this paper, we proposed a decentralized data sharing and access control mechanism leveraging the microservices and blockchain technology, called BlendMAS, to handle the challenges in IoT-based public safety system. A concept-proof prototype has been built in a physical IoT network environment to verify the feasibility. The hash index authentication and access control models are transcoded to smart contracts deployed on the private Ethereum blockchain network. The functionality of smart surveillance and security services are decoupling into separate containerized microservices and deployed on distributed edge and fog computing nodes. Extensive experimental studies have been conducted and the results are encouraging. It validated that the BlendMAS solution is able

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:37:47 UTC from IEEE Xplore. Restrictions apply.

to efficiently and effectively enforce identity authentication and access control policy in a distributed IoT-based smart surveillance network. This work has demonstrated that the proposed BlendMAS framework is a promising approach to provide a decentralized, scalable, fine-grained architectural security mechanism for SPS system.

While the reported work has shown great potential, there is still a long way to go towards a complete decentralized and lightweight security solution for IoTs and edge computing. Deeper insights are in need. Part of our on-going effort is focused on further exploration of the micro-blockchain platform based on permissioned blockchain network and lightweight consensus algorithm running on IoT devices.

ACKNOWLEDGEMENTS

The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the United States Air Force.

REFERENCES

[1] "Ethereum Homestead Documentation," http://www.ethdocs.org/en/latest/index.html.
[2] "Flask: A Pyhon Microframework," http://flask.pocoo.org/.
[3] "Go-ethereum," https://ethereum.github.io/go-ethereum/.
[4] "Solidity," http://solidity.readthedocs.io/en/latest/.
[5] "SQLite," https://www.sqlite.org/index.html.
[6] E. Blasch, É. Bossé, and D. A. Lambert, *High-level information fusion management and systems design*. Artech House, 2012.
[7] B. Butzin, F. Golatowski, and D. Timmermann, "Microservices approach for the internet of things," in *Emerging Technologies and Factory Automation (ETFA), 2016 IEEE 21st International Conference on*. IEEE, 2016, pp. 1–6.
[8] F. F. Chamasemani and L. S. Affendey, "Systematic review and classification on video surveillance systems," *International Journal of Information Technology and Computer Science (IJITCS)*, vol. 5, no. 7, p. 87, 2013.
[9] N. Chen, Y. Chen, E. Blasch, H. Ling, Y. You, and X. Ye, "Enabling smart urban surveillance at the edge," in *Smart Cloud (SmartCloud), 2017 IEEE International Conference on*. IEEE, 2017, pp. 109–119.
[10] N. Chen, Y. Chen, S. Song, C.-T. Huang, and X. Ye, "Smart urban surveillance using fog computing," in *Edge Computing (SEC), IEEE/ACM Symposium on*. IEEE, 2016, pp. 95–96.
[11] Y. Chen, E. Blasch, N. Chen, A. Deng, H. Ling, and G. Chen, "Real-time wami streaming target tracking in fog," in *Sensors and Systems for Space Applications IX*, vol. 9838. International Society for Optics and Photonics, 2016, p. 98380D.
[12] S. K. Datta and C. Bonnet, "Next-generation, data centric and end-to-end iot architecture based on microservices," in *2018 IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia)*. IEEE, 2018, pp. 206–212.
[13] T. Fuse and K. Kamiya, "Statistical anomaly detection in human dynamics monitoring using a hierarchical dirichlet process hidden markov model," *IEEE Transactions on Intelligent Transportation Systems*, 2017.
[14] M. T. Hammi, B. Hammi, P. Bellot, and A. Serhrouchni, "Bubbles of trust: A decentralized blockchain-based authentication system for iot," *Computers & Security*, vol. 78, pp. 126–142, 2018.
[15] L. F. Herrera-Quintero, J. C. Vega-Alfonso, K. B. A. Banse, and E. C. Zambrano, "Smart its sensor for the transportation planning based on iot approaches using serverless and microservices architecture," *IEEE Intelligent Transportation Systems Magazine*, vol. 10, no. 2, 2018.
[16] A. Krylovskiy, M. Jahn, and E. Patti, "Designing a smart city internet of things platform with microservice architecture," in *Future Internet of Things and Cloud (FiCloud), 2015 3rd International Conference on*. IEEE, 2015, pp. 25–30.
[17] D. Lu, D. Huang, A. Walenstein, and D. Medhi, "A secure microservice framework for iot," in *Service-Oriented System Engineering (SOSE), 2017 IEEE Symposium on*. IEEE, 2017, pp. 9–18.
[18] D. Nagothu, R. Xu, S. Y. Nikouei, and Y. Chen, "A microservice-enabled architecture for smart surveillance using blockchain technology," in *2018 IEEE International Smart Cities Conference (ISC2)*. IEEE, 2018, pp. 1–4.
[19] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.
[20] S. Y. Nikouei, Y. Chen, A. Aved, and E. Blasch, "Eiqis: Toward an event-oriented indexable and queryable intelligent surveillance system," *arXiv preprint arXiv:1807.11329*, 2018.
[21] S. Y. Nikouei, Y. Chen, S. Song, and T. R. Faughnan, "Kerman: A hybrid lightweight tracking algorithm to enable smart surveillance as an edge service," *arXiv preprint arXiv:1808.02134*, 2018.
[22] S. Y. Nikouei, Y. Chen, S. Song, R. Xu, B.-Y. Choi, and T. R. Faughnan, "Intelligent surveillance as an edge network service: from harr-cascade, svm to a lightweight cnn," *arXiv preprint arXiv:1805.00331*, 2018.
[23] ——, "Real-time human detection as an edge service enabled by a lightweight cnn," in *Edge Computing, the IEEE International Conference on*, 2018.
[24] S. Y. Nikouei, R. Xu, Y. Chen, A. Aved, and E. Blasch, "Decentralized smart surveillance through microservices platform," in *2019 SPIE Defense + Commercial Sensing*. SPIE, 2019.
[25] S. Y. Nikouei, R. Xu, D. Nagothu, Y. Chen, A. Aved, and E. Blasch, "Real-time index authentication for event-oriented surveillance video query using blockchain," in *2018 IEEE International Smart Cities Conference (ISC2)*. IEEE, 2018, pp. 1–8.
[26] M. Ribeiro, A. E. Lazzaretti, and H. S. Lopes, "A study of deep convolutional auto-encoders for anomaly detection in videos," *Pattern Recognition Letters*, 2017.
[27] M. Swan, *Blockchain: Blueprint for a new economy*. " O'Reilly Media, Inc.", 2015.
[28] N. Szabo, "Formalizing and securing relationships on public networks," *First Monday*, vol. 2, no. 9, 1997.
[29] X. Wang, "Intelligent multi-camera video surveillance: A review," *Pattern recognition letters*, vol. 34, no. 1, pp. 3–19, 2013.
[30] R. Wu, Y. Chen, E. Blasch, B. Liu, G. Chen, and D. Shen, "A container-based elastic cloud architecture for real-time full-motion video (fmv) target tracking," in *2014 IEEE Applied Imagery Pattern Recognition Workshop (AIPR)*. IEEE, 2014, pp. 1–8.
[31] R. Wu, B. Liu, Y. Chen, E. Blasch, H. Ling, and G. Chen, "Pseudo-real-time wide area motion imagery (wami) processing for dynamic feature detection," in *2015 18th International Conference on Information Fusion (Fusion)*. IEEE, 2015, pp. 1962–1969.
[32] ——, "A container-based elastic cloud architecture for pseudo real-time exploitation of wide area motion imagery (wami) stream," *Journal of Signal Processing Systems*, vol. 88, no. 2, pp. 219–231, 2017.
[33] R. Xu, Y. Chen, E. Blasch, and G. Chen, "Blendcac: A blockchain-enabled decentralized capability-based access control for iots," in *The 2018 IEEE International Conference on Blockchain (Blockchain-2018)*. IEEE, 2018, pp. 1–8.
[34] R. Xu., Y. Chen, E. Blasch, and G. Chen, "Blendcac: A smart contract enabled decentralized capability-based access control mechanism for the iot," *Computers*, vol. 7, no. 3, p. 39, 2018.
[35] R. Xu, Y. Chen, E. Blasch, and G. Chen, "Exploration of blockchain-enabled decentralized capability-based access control strategy for space situation awareness," *Optical Engineering*, vol. 58, pp. 58 – 58 – 16, 2019. [Online]. Available: https://doi.org/10.1117/1.OE.58.4.041609
[36] R. Xu, X. Lin, Q. Dong, and Y. Chen, "Constructing trustworthy and safe communities on a blockchain-enabled social credits system," in *Proceedings of the 15th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*. ACM, 2018, pp. 449–453.
[37] R. Xu, S. Y. Nikouei, Y. Chen, A. Polunchenko, S. Song, C. Deng, and T. R. Faughnan, "Real-time human objects tracking for smart surveillance at the edge," in *2018 IEEE International Conference on Communications (ICC)*. IEEE, 2018, pp. 1–6.
[38] D. Yu, Y. Jin, Y. Zhang, and X. Zheng, "A survey on security issues in services communication of microservices-enabled fog applications," *Concurrency and Computation: Practice and Experience*, p. e4436, 2018.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:37:47 UTC from IEEE Xplore. Restrictions apply.